

DSAII- HW12

Tomas Salaz

April 29, 2026

Problem 1

To generate the safest path for the trucks, we can use a combination of Dijkstra's Algorithm. We need to modify the edge weights, since they are given in Probabilities and multiplying probabilities would give a bad answer. For example, given 2 paths $P = s \rightarrow t \rightarrow u$ and $P' = s \rightarrow t' \rightarrow u$ where $w(s, t) = .9, w(t, u) = .1$ and $w(s, t') = w(t', u) = .5$, The probability of path $P = .09$ and probability of path $P' = .25$. Since Dijkstra's is used for adding the weights we need to modify the Edge weights using the log of the value to represent the probabilities correctly.

The **Algorithm** is to

- (1) Construct a new graph with the same vertices and different edge weights. $G' = (V, E)$, For $\forall(u, v) \in E$, $w(u, v) = -\log p(u, v)$.
- (2) Run Dijkstra's on G' from source s to destination t to compute the shortest path, which is the safest since the weights are normalized probability where the lower the value the safer.
- (3) Return the path given by from Dijkstra's

Correctness: For any path P , from s to t , $P = (s \rightarrow v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k \rightarrow t)$ in G , the Probability of safety for the path is given by $P(Path) = \prod_{i=1}^k P(v_{i-1}, v_i)$

Since logs are an increasing function, maximizing $P(Path)$ is the same as maximizing $\log P(Path)$ which is also equivalent to minimizing $-\log P(Path)$, hence we use Dijkstra's to minimize the path on $-\log P(Path)$ to get the safest path in G .

Runtime: Constructing G' takes $O(V + E)$ time, this accounts for the changes in edge weights. Running Dijkstra's with a binary heap takes $O((V + E) + \log V)$

Problem 2

Part A: Since there are negative weights we can use a variation of Bellman-Ford. The Algorithm for this problem is to

- (1) Create a Directed Graph $G = (V, E)$ where each vertex v_i represents a currency c_i , and for every pair of vertices, (i, j) where $i \neq j$ we add an edge from $v_i \rightarrow v_j$, with weight $w = -\log R[i, j]$, which comes from the table. Like the previous problem we need to normalize the weights so we can run a minimum

algorithm to maximize the output.

(2) We also need to add a source vertex s to every node with weight $w = 0$. This happens during the construction and is needed since Bellman-Ford is a Single Source Algo and we need to detect Negative Cycles, but we can only detect from a source, we use this s vertex to have a way in.

(3) Run Bellman-Ford on G starting at s on the the first currency in the sequence while detecting cycles. After $|V| - 1$ passes, Bellman-Ford does another pass over all the edges.

(3a) If no edge can be relaxed further, then there is no negative weight cycle since

(3b) If some edge (u, v) can be relaxed further, so if $d[u] + w(u, v) < d[v]$, a negative weight cycle exists.

(4) Return that Arbitrage exists IFF Bellman-Ford detects a negative weight cycle.

The **Runtime** of the algo take $O(n^2)$ to create the $n \times n$ table. The graph has $|V| = n + 1$ vertices and $|E| = n^2 + n$ edges. Bellman-Ford runs in $O(|V| + |E|)$ time, so $O(n \times n^2) = O(n^3)$ total time.

Part B: (1) Run the same Bellman-Ford algo while maintaining a predecessor array, P where $P[v]$ stores v 's predecessor.

(2) If an edge (u, v) can still be relaxed, let $x = v$ and follow the the predecessor pointer $|V|$ times, guarantees that x is in the negative weight cycle.

(3) Let $y = x$ and follow the pointers until y is detected again, recording the sequence. This sequence reversed is the Arbitrage Cycle of Currency's

The **Runtime** for this algo is still $O(n^3)$ from Bellman Ford, the path reconstruction can be done in the final check of Bellman-Ford and thus is still $O(n^3)$

Problem 3

Let **SATSolver** be the polynomial time algo that decides whether a 3CNF formula is satisfiable. Given a 3CNF formula T , over variables $x_1, x_2, \dots, x_n$. For each x_i , we process the variables one at a time as follows:

(1) Construct T' by using T and letting $x_i = TRUE$. Thus, every clause containing x_i is TRUE, so we remove those clauses. We also remove all clauses containing $\neg x_i$.

(2) Call **SATSolver**(T')

(3) If **SATSolver**(T') = *TRUE*, then $x_i = TRUE$, and we replace T with T'

(4) If **SATSolver**(T') = *FALSE*, then we replace T with the formula given by T when $x_i = FALSE$ instead.

(5) Once all x_i have been processed, output the assignment of each value, $(x_1, x_2, \dots, x_n)$

Correctness by Induction on the number of Variables processed that the formula maintained in the algo remains satisfiable.

BC: Before any processing, the formula T is trivially satisfiable.

IH: After processing the first i variables, the maintained formula with $(x_1, x_2, \dots, x_i)$ fixed, is satisfiable.

IS: Assuming the current formula T is satisfiable and considering processing x_i , Let T_{true} and T_{false} be the formulas given by setting x_i to True or False. Since T is satisfiable then every x_i assignment must be True or False, thus either T_{true} or T_{false} is satisfiable, which is determined by **SATSolver**.

After processing all n variables, the maintained formula is satisfiable and contains no variables such that every clause has been satisfied and removed. Therefore, the recorded assignment from (5) satisfies the original formula T .

Runtime: The Algo makes n calls to **SATSolver**, each call is on a formula of at most size $|T|$. The construction of each simplified formula at every step take $O(|T|)$ time. Let $S(m)$ be the runtime of **SATSolver** on size m , so it is a polynomial. The Total runtime is given by $O(n \times S(|T|) + |T|)$, which is also a polynomial.

Problem 4

To prove that the Radio Towers problem is NP-Hard, we can start by using the **k-coloring** problem which is NP-Hard for $k \geq 3$. The **k-coloring** problem is defined as follows: Given an undirected graph $G = (V, E)$ and an integer k , determine whether we can color every vertex in G such that for every edge, (u, v) , $Color(u) \neq Color(v)$.

Reduction: Given an instance (G, k) , of **k-coloring**, we can construct an instance of **Radio-Towers**, (S, T, k) as follows:

- (1) Let $V = S$, so there is one vertex in G for each tower in **Radio-Towers**
 - (2) Let $T = \{(u, v) \in E\}$, so each edge in G is the corresponding endpoints as a subset of T .
 - (3) Let k be the number of allowed Radio Frequency's in **Radio Towers**
- The Construction takes $O(|V| + |E|)$ time, which is polynomial in the size of the input.

For **Correctness** we must show that G has a valid **k-coloring** IFF the **Radio Towers** instance has a valid frequency assignment.

- (1) Forward Direction: Assuming G has a valid **k-coloring**, then color $c : V \rightarrow \{1, 2, \dots, k\}$. We defined frequency assignments $f : S \rightarrow \{1, 2, \dots, k\}$. This implies that $f(v) = c(v)$ for every Tower $v \in S$. Consider any subset $\{u, v\} \in T$, then by construction, The subset T corresponds to an edge $(u, v) \in E$. Since c is a valid coloring, $c(u) \neq c(v)$, thus $f(u) \neq f(v)$. Therefore every subset in T must have different frequency's, do f is a valid **Radio Towers** solution.
- (2) Backwards Direction: Assuming there is a valid **Radio Towers** solution, then $f : S \rightarrow \{1, 2, \dots, k\}$. This implies that color $c : V \rightarrow \{1, 2, \dots, k\}$ for every vertex in G . Consider any edge in $(u, v) \in E$, then by construction, the subset $\{(u, v)\}$ belongs to T and f . Since f is a valid frequency assignment, then $f(u) \neq f(v)$ and thus $c(u) \neq c(v)$. Therefore there c is a valid solution for **k-coloring**.
- (3) The reduction from **k-coloring** to **Radio Towers** is valid since every TRUE instance of **Radio Towers** is a valid in **k-coloring** and vice versa. Therefore, both problems are NP-Hard